



CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Minor Project On Real-Time Disaster Notifier

SUBMITTED BY:

ROHAN & NARESHDEEP

UID – 23BCA10072 & 23BCA10134

SECTION - 9A

SUBMITTED TO:

MR. VIKAS RAHEJA

SIGNATURE:

Title: Real-Time Disaster Notifier

Introduction

Disasters like earthquakes, floods, and cyclones occur unexpectedly, causing huge damage to life and property. The Real-Time Disaster Notifier is a web/mobile-based application designed to provide instant alerts and updates about natural or man-made disasters occurring in a user's region.

The main goal is to detect, collect, and notify users in real-time about potential hazards using APIs, sensors, or verified government data sources (e.g., NDMA, IMD, or NASA). It helps people stay informed and take preventive measures quickly.

Objective

The purpose of this project is to create a **web-based application** that provides users with **real-time weather updates and disaster alerts** based on their current or chosen location. It helps people stay informed about potential natural disasters or unsafe weather conditions like **storms, heavy rain, or air pollution**.

Working Principle

1. **User Input:**
The user enters a city name (e.g., *London*).
2. **API Call:**
JavaScript sends a request to the WeatherAPI using the `fetch()` function:
3. `fetch('https://api.weatherapi.com/v1/current.json?key=API_KEY&q=city&aqi=yes')`
4. **Data Processing:**
The app extracts:
 - Temperature
 - Weather condition (e.g., "Storm", "Sunny", "Rain")
 - Air Quality Index (AQI)

Scope of the Project

This project can be useful for:

Citizens in disaster-prone areas.

Disaster management authorities.

Travelers and logistics companies.

Government and NGOs involved in emergency response.

Disaster Notification:

Based on conditions:

- If storm/rain detected → “Heavy Weather Alert!”
- If AQI > 100 → “Poor Air Quality Alert”
- Else → “Weather is Normal”

System Architecture

Input:

Live disaster data from APIs (e.g., OpenWeatherMap, USGS Earthquake API, IMD feeds).

Location input from user (GPS or manual).

Process:

Fetch and process disaster alerts.

Match alert location with the user’s current location.

Trigger real-time notification if a disaster is nearby.

Output:

Display alert on dashboard.

Send push/email/SMS notification.

Provide safety instructions.

Tools & Technologies Used

Technology	Purpose
HTML5	Structure of the web app
CSS3	Designing and styling the UI
JavaScript (ES6)	Fetching data and updating UI dynamically
WeatherAPI	Provides real-time weather and air quality data
VS Code / CodePen Development Environment	

Algorithm / Workflow

User logs into the system.

System fetches user's location (latitude & longitude).

Disaster data is fetched via API.

If any disaster coordinates match user location radius → Alert generated.

Notification sent with safety instructions and contact details.

Sample APIs

Earthquake Data (USGS):

https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_hour.geojson

Weather Alert (OpenWeatherMap):

https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}

Output Example

User Input: "Delhi"

Output:

- Temperature: 31°C
- Condition: Thunderstorm
- Alert: "Heavy weather alert! Stay safe."

Future Scope

Integration with IoT sensors for on-ground detection.

AI-based disaster prediction using historical data.

Multilingual support for rural areas.

Integration with government alert systems.

Conclusion

The Real-Time Disaster Notifier project enhances disaster management by providing timely and reliable alerts. It empowers individuals to act quickly and can save lives during emergencies. With further development, it could be expanded into a large-scale public safety system.

References

USGS Earthquake API

OpenWeatherMap API

NDMA India

IMD India

CODE -

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
<title>☁ Real-Time Disaster Notifier</title>
```

```
<style>
```

```
* {
```

```
margin: 0;
```

```
padding: 0;
```

```
box-sizing: border-box;

font-family: "Poppins", sans-serif;

}
```

```
/* Animated Gradient Background */
```

```
body {

  min-height: 100vh;

  display: flex;

  align-items: center;

  justify-content: center;

  background: linear-gradient(270deg, #1f4037, #185a9d, #43cea2, #99f2c8);

  background-size: 600% 600%;

  animation: gradientShift 10s ease infinite;

  overflow: hidden;

}
```

```
@keyframes gradientShift {

  0% {

    background-position: 0% 50%;

  }

  50% {

    background-position: 100% 50%;

  }

  100% {

    background-position: 0% 50%;

  }

}
```

```
/* Floating Weather Icons Animation */  
  
.floating {  
    position: absolute;  
    top: 10%;  
    left: 5%;  
    width: 100px;  
    opacity: 0.15;  
    animation: float 6s ease-in-out infinite alternate;  
}
```

```
@keyframes float {  
    from {  
        transform: translateY(0px);  
    }  
    to {  
        transform: translateY(20px);  
    }  
}
```

```
.container {  
    background: rgba(255, 255, 255, 0.2);  
    backdrop-filter: blur(20px);  
    border-radius: 25px;  
    box-shadow: 0 8px 40px rgba(0, 0, 0, 0.3);  
    max-width: 420px;  
    width: 100%;  
    padding: 40px 25px;  
    color: #fff;
```

```
text-align: center;
position: relative;
overflow: hidden;
transition: transform 0.3s ease;
}
```

```
.container:hover {
  transform: translateY(-5px);
}
```

```
h1 {
  font-size: 2rem;
  margin-bottom: 15px;
  letter-spacing: 1px;
  background: linear-gradient(90deg, #ffffff, #b9fff8);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  text-shadow: 0 0 10px rgba(255, 255, 255, 0.3);
}
```

```
input {
  width: 85%;
  padding: 12px;
  border: none;
  border-radius: 12px;
  margin: 15px 0;
  outline: none;
  font-size: 1rem;
```

```
text-align: center;

color: #333;

transition: 0.3s;

}
```

```
input:focus {

  box-shadow: 0 0 10px rgba(255, 255, 255, 0.6);

}
```

```
button {

  padding: 12px 24px;

  border: none;

  border-radius: 10px;

  background: linear-gradient(135deg, #6dd5ed, #2193b0);

  color: white;

  cursor: pointer;

  transition: all 0.4s ease;

  font-size: 1rem;

  font-weight: 500;

}
```

```
button:hover {

  background: linear-gradient(135deg, #2193b0, #6dd5ed);

  transform: scale(1.08);

  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.3);

}
```

```
.weather-box {
```

```
margin-top: 25px;
background: rgba(255, 255, 255, 0.15);
border-radius: 15px;
padding: 25px;
box-shadow: inset 0 0 10px rgba(255, 255, 255, 0.2);
animation: fadeIn 1s ease;
display: none;
}
```

```
.weather-box img {
width: 100px;
filter: drop-shadow(0px 0px 5px rgba(255, 255, 255, 0.7));
}
```

```
.temp {
font-size: 2.3rem;
margin: 10px 0;
font-weight: bold;
text-shadow: 0 0 10px rgba(255, 255, 255, 0.2);
}
```

```
.condition {
font-size: 1.3rem;
text-transform: capitalize;
opacity: 0.9;
}
```

```
.alert {
```

```
margin-top: 15px;
padding: 12px;
border-radius: 10px;
background-color: rgba(255, 80, 80, 0.35);
font-weight: bold;
display: none;
box-shadow: 0 0 10px rgba(255, 80, 80, 0.4);
animation: pulse 1.5s infinite alternate;
}
```

```
@keyframes pulse {
  0% {
    transform: scale(1);
  }
  100% {
    transform: scale(1.03);
  }
}
```

```
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

```
}
```

```
footer {  
  margin-top: 25px;  
  font-size: 0.85rem;  
  opacity: 0.8;  
  color: #f0f0f0;  
}
```

```
@media (max-width: 480px) {  
  .container {  
    padding: 25px;  
  }  
  h1 {  
    font-size: 1.6rem;  
  }  
}  
</style>  
</head>  
<body>  
  <!-- Floating Icons -->  
    
  
```

```
<div class="container">
```

```
<h1><img alt="Cloud icon" data-bbox="191 238 215 253"/> Real-Time Disaster Notifier</h1>
```

```
<input
```

```
  type="text"
```

```
  id="locationInput"
```

```
  placeholder="Enter city name (e.g. Delhi)"
```

```
</div>
```

```
<button onclick="getWeather()">Check Weather</button>
```

```
<p id="clock"></p>
```

```
<div class="weather-box" id="weatherBox">
```

```
  <img id="icon" src="" alt="Weather Icon" />
```

```
  <div class="temp" id="temperature"></div>
```

```
  <div class="condition" id="condition"></div>
```

```
  <div class="alert" id="alertBox"></div>
```

```
</div>
```

```
<footer>Powered by WeatherAPI | Designed with <img alt="Heart icon" data-bbox="558 685 582 700"/></footer>
```

```
</div>
```

```
<script>
```

```
  // Clock
```

```
  setInterval(() => {
```

```
    const now = new Date();
```

```
    document.getElementById("clock").innerText = now.toLocaleString();
```

```
}, 1000);
```

```
async function getWeather() {
```

```
    const city = document.getElementById("locationInput").value.trim();
```

```
    const weatherBox = document.getElementById("weatherBox");
```

```
    const alertBox = document.getElementById("alertBox");
```

```
    if (!city) {
```

```
        alert("Please enter a location!");
```

```
        return;
```

```
    }
```

```
    const apiKey = "1622010d93c54ba0ac0163059250211";
```

```
    const url =
```

```
    `https://api.weatherapi.com/v1/current.json?key=${apiKey}&q=${city}&aqi=yes`;
```

```
    try {
```

```
        const response = await fetch(url);
```

```
        if (!response.ok) throw new Error("Location not found");
```

```
        const data = await response.json();
```

```
        const temperature = data.current.temp_c;
```

```
        const condition = data.current.condition.text;
```

```
        const icon = data.current.condition.icon;
```

```
        const aqi = data.current.air_quality.pm2_5;
```

```
        document.getElementById("icon").src = "https:" + icon;
```

```
        document.getElementById("temperature").innerHTML = `${temperature}°C`;
```

```
document.getElementById("condition").innerHTML = condition;
```

```
let alertMsg = "";
```

```
if (
```

```
    condition.toLowerCase().includes("storm") ||
```

```
    condition.toLowerCase().includes("rain")
```

```
) {
```

```
    alertMsg = "⚠️ Heavy weather alert! Stay safe.";
```

```
    alertBox.style.backgroundColor = "rgba(255, 50, 50, 0.5)";
```

```
} else if (aqi > 100) {
```

```
    alertMsg =
```

```
        "🌫️ Poor air quality detected. Avoid outdoor activities.";
```

```
    alertBox.style.backgroundColor = "rgba(255, 165, 0, 0.5)";
```

```
} else {
```

```
    alertMsg = "✅ Weather conditions are normal.";
```

```
    alertBox.style.backgroundColor = "rgba(50, 205, 50, 0.5)";
```

```
}
```

```
alertBox.innerHTML = alertMsg;
```

```
alertBox.style.display = "block";
```

```
weatherBox.style.display = "block";
```

```
} catch (error) {
```

```
    alertBox.style.display = "none";
```

```
    weatherBox.style.display = "none";
```

```
    alert(
```

```
        "❌ Unable to fetch data. Please check city name or try again later."
```

```
    );
```

```
}
```

```

    }

    window.onload = function () {
        if (navigator.geolocation) {
            navigator.geolocation.getCurrentPosition(async function (position) {
                const lat = position.coords.latitude;
                const lon = position.coords.longitude;
                const apiKey = "1622010d93c54ba0ac0163059250211";
                const url =
`https://api.weatherapi.com/v1/current.json?key=${apiKey}&q=${lat},${lon}&aqi=yes`;

                const response = await fetch(url);
                const data = await response.json();
                document.getElementById("locationInput").value = data.location.name;
                getWeather();
            });
        }
    };
</script>
</body>
</html>

```

How It Works

User enters a city name.

The app fetches weather data from WeatherMap API.

Based on the condition (rain, storm, etc.), it shows a real-time alert.

Project Report Summary

Project Title: Real-Time Disaster Notifier

Objective: To alert users about real-time weather or disaster risks using live API data.

Tools Used: HTML, CSS, JavaScript

Output: Displays instant alerts for heavy rain, storms, or clear weather.

Advantages:

Simple and lightweight.

Uses real-time API data.

Easy to expand for more disaster types.